🚀 Flet-base Template

A premium unofficial starter kit for building multi-platform applications with Python and Flet.

🛠️ Initializing a New Project

This repository serves as a **template** for new Flet-based apps. You can start from scratch using Flet's own project generator and then layer in the custom modules that live here.

1. **Create the base application**

   ```
   flet create my_app
   cd my_app
   ```

   This will generate a minimal Flet project structure (including an `app.py` or similar entry point) and install the core `flet` package.

2. **Flet-base Package Structure & Global Config**

   All utility modules (`components`, `layout`, `router`, `themes`, `translations`) are now neatly packed under the `flet_base` package namespace to prevent import conflicts and make publishing to PyPI straightforward.

   Before starting the app, you can easily configure defaults using the global `config.py` object:

   ```
   from flet_base.config import flet_config

   flet_config.default_language = "es"
   flet_config.default_theme_mode = "dark"
   flet_config.font_files = {
       "Inter": "assets/fonts/Inter-Regular.ttf",
       "Inter Bold": "assets/fonts/Inter-Bold.ttf",
   }
   flet_config.main_font_family = "Inter"  # None keeps Flet default font
   # etc...
   ```

3. **Install as a Package**

   You can install the local modules as an editable package:

   ```
   pip install -e .
   ```

It is also available in pip.

🌍 Translation Manager

A robust and simple utility to manage multi-language support in **Flet** applications. This manager handles CSV-based translations, automatic language detection, and user preference persistence.

🚀 Key Features

- **CSV-Powered**: Manage all your strings in a single, easy-to-edit CSV file.
- **Auto-Detection**: Automatically initializes with the user's system language (`page.locale`).
- **Persistence**: Integrates with `page.shared_preferences` to remember the user's language choice.
- **Safe Failover**: Provides recursive fallback (Current Language → Default Language → Key Name).
- **Bidirectional Mapping**: Supports converting between ISO codes (e.g., `en`) and human-readable names (e.g., `English`).

> [!IMPORTANT] The `translations_csv_path` in `flet_config` is **vital**. You must provide an absolute path to your translations CSV for the manager to function. If not set, translations will not be loaded.

🛠️ Internal Structure & API

Core Methods

`TranslationManager(csv_path=None, default_lang="en", csv_separator=",")`

Creates a new manager instance.

- **`csv_path`** : Optional. Path to your translation file. If omitted, the manager uses `flet_config.translations_csv_path` . If neither is configured, the manager will start empty safely and log a warning to the console.
- **`default_lang`** : Optional. Override the `flet_config.default_language` fallback.
- **`csv_separator`** : Optional. Override the `flet_config.translations_csv_separator` fallback (default is `","`).

`awake(page: ft.Page)`

The essential initialization hook for Flet apps.

1. Checks `page.shared_preferences` for a saved language.
2. If not found, attempts to use `page.locale`.
3. Falls back to `default_lang` if neither is available.

`translate(key: str) -> str`

The main method for UI strings.

- Usage: `tm.translate("welcome_message")`

`set_language(lang_code: str)`

Updates the active language. Note: This changes the internal state; you should also update `page.shared_preferences` to persist the change.

Utility Helpers

- `get_available_languages()`: Returns a list of readable names (e.g., `["English", "Español"]`).
- `get_language_name(code)`: Static method. `en` -> `English`.
- `get_language_code(name)`: Static method. `Español` -> `es`.

📊 CSV Format Requirement

Your `translations.csv` must follow this header structure:

key	en	es	fr	...
hello	Hello	Hola	Bonjour	...
login_btn	Login	Iniciar Sesión	Connexion	...

> [!TIP] Use standard ISO 639-1 codes (2 letters) for column headers to ensure compatibility with `page.locale`.

💡 Implementation Example

```python
import flet as ft
from flet_base.config import flet_config

# 0. Point to your CSV file before taking translations
flet_config.translations_csv_path = "assets/translations.csv"
flet_config.translations_csv_separator = "," # Optional

from flet_base.translations import instance_translation_manager as tm


async def main(page: ft.Page):
    # 1. Initialize & Awake
    await tm.awake(page)

    # 2. Reactive UI Component
    user_greeting = ft.Text(tm.translate("hello"), size=30, weight="bold")

    async def change_lang(code):
        await tm.set_language(code)
        page.shared_preferences.set("language", code)
        # Update the UI
        user_greeting.value = tm.translate("hello")
        page.update()

    page.add(
        user_greeting,
        ft.ElevatedButton(
            "Change to Spanish",
            on_click=lambda _: await change_lang("es")
        )
    )

ft.app(target=main)
```

🧪 Testing

The `translations.py` file includes a built-in test block. You can run it directly to verify your CSV loading:

```
python flet_base/translations/translations.py
```

🎨 Theme Manager

A lightweight helper for toggling between light and dark palettes in Flet applications. Themes are stored in `page.shared_preferences` so that user choices survive app restarts. The API mirrors the translation manager for consistency.

🛠️ Key Features

- **Light & Dark palettes** with sensible defaults but easy to override.
- **Persistent preference** using `page.shared_preferences` under the key `"theme"`.
- **Automatic initialization** on page creation (`awake()`).
- **Convenience methods**: `set_light_theme` , `set_dark_theme` , `switch_theme` .

🛠️ Internal Structure & API

`themes(default_theme=light_theme)`

Constructor taking a starting palette (either `light_theme` or `dark_theme`).

`awake(page: ft.Page)`

Reads stored preference and applies the corresponding palette; if none is found it consults `ft.ThemeMode.SYSTEM` and the provided default. The order is:

- Checks `page.shared_preferences` for a saved theme.
- If not found, attempts to use system theme.
- Falls back to `default` if neither is available.

`set_light_theme(page)` / `set_dark_theme(page)`

Apply a specific palette and save the choice. They also update `actual_theme` so your code can query the active color values.

`switch_theme(page)`

Toggle between light and dark. If the current palette is neither, it falls back to `awake()` behaviour.

Even if you change theme, app's border might not change color because it is fixed in system configuration, for example in windows.

🛠️ Change theme colors

> [!IMPORTANT] Custom themes MUST follow the key structure shown below. UI components (like buttons and texts) rely on these specific keys (e.g., `primary` , `on_primary` , `text_color` , etc.) to style themselves correctly. Removing keys will break the UI components.

You can modify themes globally using `flet_config`:

```
from flet_base.config import flet_config

# Modify existing keys
flet_config.light_theme["primary"] = "#6200EE"
flet_config.light_theme["text_color"] = "#000000"

# Or provide a whole new dictionary (ensure all keys are present)
flet_config.dark_theme = {
    "primary": "#BB86FC",
    "on_primary": "#000000",
    "secondary": "#03DAC6",
    "on_secondary": "#000000",
    "background": "#121212",
    "on_background": "#FFFFFF",
    "surface": "#1E1E1E",
    "on_surface": "#FFFFFF",
    "text_color": "#FFFFFF",
    "error": "#CF6679",
    "on_error": "#000000",
    "warning": "#FFB300",
    "success": "#66BB6A",
    "link": "#5252FF",
}
```

💡 Theme Example

```python
import flet as ft
from flet_base.themes import instance_themes as themes


def main(page: ft.Page):
    th = themes()
    th.awake(page)

    txt = ft.Text("Hello, world!", size=30)

    def on_click(_):
        th.switch_theme(page)
        # update UI colors (demo uses the palette directly)
        txt.color = th.actual_theme["primary"]
        page.update()

    page.add(
        txt,
        ft.ElevatedButton("Toggle theme", on_click=on_click),
    )

ft.app(target=main)
```

🔤 Typography & Custom Fonts

You can configure custom fonts globally using `flet_config` and apply one as your main UI font family. This affects all text helpers and component labels (including button text).

```python
from flet_base.config import flet_config

# Optional: register multiple font files (local paths or URLs)
flet_config.font_files = {
    "Inter": "assets/fonts/Inter-Regular.ttf",
    "Inter SemiBold": "assets/fonts/Inter-SemiBold.ttf",
}

# Main family used across components.
# Keep None to use the default Flet font.
flet_config.main_font_family = "Inter"
```

> [!TIP] Set typography options before calling `themes.awake(page)` or building your UI so the font settings are applied from the beginning.

⌨️ Keyboard Shortcut Manager

A modular and OS-agnostic system to handle keyboard events. It allows you to define shortcuts like `mod+s` which automatically resolves to `Ctrl+s` on Windows/Linux and `Cmd+s` on macOS.

🚀 Key Features

- **OS-Agnostic**: Use `mod` alias for cross-platform compatibility.
- **Direct Mapping**: Map key combinations directly to sync or async functions.
- **Async Support**: Automatically handles asynchronous callbacks.
- **Conflict Detection**: Warns if multiple actions are assigned to the same shortcut.
- **Robust Normalization**: Handles platform-specific keys (like `cmd`, `win`, `command`) and prevents duplicate modifier detection.

🛠️ API Reference

`ShortcutManager()`

Creates a new manager instance.

`register(shortcut_str, callback)`

Registers a single shortcut.

- **`shortcut_str`**: String representation e.g., `"mod+s"`, `"ctrl+shift+p"`, `"alt+f4"`.
- **`callback`**: The function (sync or async) to execute.

`register_many(shortcuts_dict)`

Registers multiple shortcuts at once from a dictionary mapping shortcut strings to functions.

`handle_event(e: ft.KeyboardEvent)`

The primary event handler to be connected to `page.on_keyboard_event`.

💡 Implementation Example

```python
import flet as ft
from flet_base.keyboard_shortcuts import ShortcutManager
from flet_base.config import DEFAULT_KEY_CONFIG

async def main(page: ft.Page):
  manager = ShortcutManager()

  # 1. Register from global config (if defined as dict of callables)
  manager.register_many(DEFAULT_KEY_CONFIG)

  # 2. Register manually
  async def on_save():
      print("Saving progress...")

  manager.register("mod+s", on_save)
  manager.register("mod+shift+p", lambda: print("Command Palette Open"))

  # 3. Connect to page
  page.on_keyboard_event = manager.handle_event

  page.add(ft.Text("Press Ctrl+S (or Cmd+S on Mac) to trigger actions."))

ft.app(main)
```

🔘 Components: Buttons

The `components/buttons.py` module provides a set of pre-styled button helpers that automatically integrate with the active theme. This ensures visual consistency across your application with minimal code.

🚀 Key Features

- **Theme Integration**: Buttons automatically use colors from `themes.actual_theme`. **Use themes is required to use any component.**
- **Simplified API**: Quick creation of common button styles (filled, icon-only, text-only).
- **State Management**: Easy toggling of the `enabled` state.

🛠️ API Reference

`filled_btn(text, icon=None, on_click=None, enabled=True)`

Creates a standard filled button using the theme's `primary` color for the background and `on_primary` for the content.

`icon_filled_btn(icon, on_click=None, enabled=True)`

An icon-only version of the filled button.

`icon_btn(icon, on_click=None, enabled=True)`

A transparent background button that uses the `primary` color for the icon itself.

`text_btn(text, icon=None, on_click=None, enabled=True)`

A flat button without a background, useful for secondary actions or navigation links. Uses the theme's `text_color`.

`btn(text, icon=None, on_click=None, enabled=True, ...)`

A more flexible filled button that allows you to specify custom light and dark mode colors manually.

🧪 Usage Example

```python
import flet as ft
from flet_base.components.buttons import filled_btn, icon_btn, text_btn
from flet_base.themes import instance_themes as themes

async def main(page: ft.Page):
    await themes.awake(page)

    page.add(
        filled_btn("Submit", on_click=lambda _: print("Submit clicked!")),
        icon_btn(ft.Icons.DELETE, on_click=lambda _: print("Delete clicked!")),
        text_btn("Cancel", on_click=lambda _: page.window_close()),
    )

ft.app(target=main)
```

📝 Components: Texts

The `components/texts.py` module provides a variety of text styles and helpers that seamlessly integrate with your application's theme.

🚀 Key Features

- **Theme Consistency**: All text components automatically use the appropriate colors from your theme (e.g., `text_color`, `primary`, `secondary`, `error`).
- **Semantic Styling**: Pre-defined styles for titles, subtitles, body text, and captions.
- **Rich Content**: Support for Markdown and clickable links.

🛠️ API Reference

Titles & Subtitles

- `title(text, ...)`: Main title style using the theme's default text color.
- `title_primary(text, ...)`: Title using the `primary` color.
- `title_secondary(text, ...)`: Title using the `secondary` color.
- `subtitle(text, ...)`: Standard subtitle style.
- `subtitle_primary(text, ...)` / `subtitle_secondary(text, ...)`: Subtitles with theme colors.

Body & Utilities

- `body(text, ...)`: Standard body text.
- `caption(text, ...)`: Small text for descriptions or notes.
- `error_text(text, ...)`: Themed red text for error messages.
- `markdown(md, ...)`: Renders markdown content with theme-aware styling.
- `link(url, page, text=None, ...)`: Creates a clickable `TextButton` that opens a URL.

🧪 Usage Example

```python
import flet as ft
from flet_base.components.texts import title, subtitle, body, link
from flet_base.themes import instance_themes as themes

async def main(page: ft.Page):
    await themes.awake(page)

    page.add(
        title("Welcome to Flet-base"),
        subtitle("A premium starter kit"),
        body("This is a standard body text using the theme's colors."),
        await link("https://flet.dev", page, "Visit Flet Website")
    )

ft.app(target=main)
```

📊 Components: Data Display

The `components/data_display.py` module includes components for presenting data and media, all styled to match your theme.

🚀 Key Features

- **Interactive Data**: Simplified `DataTable` integration.
- **Visual Feedback**: Themed progress bars and loading indicators.
- **Media Support**: Easy icon and image helpers.
- **Layout Helpers**: Collapsible panels and themed material cards.

🛠️ API Reference

`datatable(columns, rows, ...)`

A themed version of `DataTable2` with pre-configured spacing and header colors.

`icon(icon, color=None, size=24)`

Creates an icon that defaults to the theme's `primary` color.

`image(src, width=100, height=100, border_radius=5)`

A simple image helper with a default border radius.

`progress_bar(value, ...)`

A linear progress bar styled with theme colors (`primary` for progress, `surface` for background).

`loading_indicator(size=50)`

A circular progress ring (loading spinner) using the theme's `primary` color.

`expansion_panel(header, content=[], expanded=False)`

A collapsible panel with a `surface`-themed background. Wrap one or more inside a `ft.ExpansionPanelList` to allow only one open at a time.

Parameter	Type	Default	Description
`header`	`str`	—	Label shown in the collapsed header bar.
`content`	`list`	`[]`	Flet controls rendered inside the expanded body.
`expanded`	`bool`	`False`	Whether the panel starts already open.

`card(content=[], color=None)`

A material card with padding and an optional custom background color. Defaults to the theme's `primary` color.

Parameter	Type	Default	Description
`content`	`list`	`[]`	Flet controls placed inside the card body.
`color`	`str | None`	`None`	Card background. Falls back to `primary` if omitted.

🧪 Usage Example

```python
import flet as ft
from flet_base.components.data_display import (
    icon, progress_bar, loading_indicator,
    expansion_panel, card,
)
from flet_base.components.texts import body, caption
from flet_base.themes import instance_themes as themes

async def main(page: ft.Page):
    await themes.awake(page)

    page.add(
        # Basic display
        icon(ft.Icons.FAVORITE),
        progress_bar(value=0.7),
        loading_indicator(size=30),

        # Collapsible panel
        ft.ExpansionPanelList(
            controls=[
                expansion_panel(
                    header="Details",
                    content=[body("Panel content goes here.")],
                    expanded=True,
                ),
            ]
        ),

        # Material card
        card(
            content=[
                body("A themed card"),
                caption("Optional sub-text."),
            ]
        ),
    )

ft.app(target=main)
```

⌨️ Components: Inputs

The `components/inputs.py` module provides a wide range of themed input controls, from basic text fields to complex date and color pickers.

🚀 Key Features

- **Theme Integrated**: All inputs use the active theme's `primary` and `background` colors.
- **Extended Functionality**: Includes built-in helpers for Pickers (Date, Time, Color).
- **Easy Configuration**: Simplified parameters for common use cases.

🛠️ API Reference

Text & Selection

- `text_input(placeholder, ...)`: A customized `TextField` with support for passwords and multiline.
- `dropdown(label, options, ...)`: A themed `Dropdown` menu.
- `checkbox(label, ...)` / `switch(label, ...)`: Boolean selection inputs with theme-aware active colors.
- `slider(label, ...)`: A themed range selector.

Pickers

- `color_picker(color="#FFFFFF", ...)`: A full-featured color selection dialog.
- `date_picker(...)` / `date_range_picker(...)`: Themed date selection utilities.
- `time_picker(...)`: A themed clock-style time picker.

🪟 Components: Modals & Overlays

The `components/modals.py` module provides simplified ways to show `AlertDialogs` and `BottomSheets` with consistent styling and dimensions.

🚀 Key Features

- **Consistent Sizing**: Default width of 700px for a premium look on large screens.
- **Themed Backgrounds**: Automatically uses the theme's `surface` color.
- **Simplified Actions**: Modal comes with a pre-configured "Close" button using `filled_btn`.

🛠️ API Reference

`modal(title_str, content, on_dismiss=None, width=700, actions=None)`

Creates an `ft.AlertDialog`.

Parameter	Type	Default	Description
`title_str`	`str`	—	Title text for the modal.
`content`	`list`	—	List of controls to show in the body.
`width`	`int`	`700`	Width of the internal container.
`actions`	`list`	`[Close Button]`	List of buttons at the bottom.

`bottom_sheet(content, width=700)`

Creates an `ft.BottomSheet` anchored to the bottom of the page.

🧪 Usage Example

```
from flet_base.components.modals import modal, bottom_sheet
from flet_base.components.texts import body

# To show a Modal
page.show_dialog(
    modal(
        "Confirmation",
        [body("Are you sure you want to proceed?")]
    )
)

# To close a Modal or Bottom Sheet
page.pop_dialog()
```

📐 Layout: Responsive Auto Layout

The `layout/responsive_auto_layout.py` module provides a specialized layout component that automatically arranges its children into rows based on available width, applying precise scaling when content exceeds the boundaries.

🚀 Key Features

- **Real-time Measurement**: Attempts to measure actual rendered widths using `on_resize` events for high accuracy.
- **Intelligent Grouping**: Groups elements into rows dynamically using a greedy algorithm.
- **Proportional Scaling**: If a group doesn't fit, it scales the entire group (width and height) down to fit the available space exactly.
- **Threshold Mode**: Forces a single-column layout when the page width drops below a customizable `threshold`.
- **Centered Alignment**: Keeps all rows horizontally and vertically centered within the layout container.

🛠️ API Reference

`ResponsiveAutoLayout(content, page, spacing=10, threshold=600)`

Parameter	Type	Default	Description
`content`	`list[ft.Control]`	—	The list of controls to be managed by the layout.
`page`	`ft.Page`	—	The active Flet page (required for resize events and padding calculation).
`spacing`	`int`	`10`	Horizontal and vertical gap between elements.
`threshold`	`int`	`600`	Width in pixels below which the layout forces a single column.

`control` (Property)

Returns the root Flet control (a `Container`) that should be added to your page or parent container.

🧪 Usage Example

```python
import flet as ft
from flet_base.layout.responsive_auto_layout import ResponsiveAutoLayout

def main(page: ft.Page):
    page.padding = 20

    # Create several cards with different widths
    cards = [
        ft.Container(width=200, height=150, bgcolor="amber", content=ft.Text("Card 1")),
        ft.Container(width=400, height=150, bgcolor="blue", content=ft.Text("Card 2")),
        ft.Container(width=150, height=150, bgcolor="green", content=ft.Text("Card 3")),
    ]

    # Initialize the layout
    ral = ResponsiveAutoLayout(
        content=cards,
        page=page,
        spacing=15,
        threshold=500
    )

    # Add it to the page
    page.add(ral.control)

ft.app(target=main)
```

🛣️ Flet Router Manager

The `router` module provides a powerful, decorator-based routing system for Flet applications. It supports dynamic route matching, query string parsing, global/route-specific middlewares, UI shells (layouts), and custom 404 handling.

🚀 Key Features

- **Express/FastAPI Style**: Use `@app.page("/route")` to define views.
- **Dynamic routes & Query Strings**: Easily capture parameters like `/user/:id` and `?mode=edit`.
- **Middlewares**: Intercept navigation globally or per-route for authentication, logging, etc.
- **Shells (Layouts)**: Wrap pages in common UI elements (AppBars, NavigationBars) based on path prefixes.

- **Shared State**: Pass data safely between routes using a built-in `shared` dictionary.
- **Navigation History**: Go back easily using built-in history tracking.

🛠️ Initialization & Setup

To use the router, instantiate `FletRouter` and pass it to `app.run()`. This replaces the standard `ft.app(target=main)` call.

```
import flet as ft
from flet_base.router import FletRouter, DataSystem

app = FletRouter(
    route_init="/login",  # Starting route
    route_login="/login",  # Auto-redirect for protected routes
)

# ... define pages ...

if __name__ == "__main__":
    app.run() # Starts the Flet app automatically
```

📄 Defining Pages

Use the `@app.page` decorator. The function must accept a `DataSystem` object and return an `ft.View`.

```
@app.page("/home", title="Main Menu", protected=True)
def home_page(data: DataSystem) -> ft.View:
    return ft.View(
        route="/home",
        controls=[
            ft.Text("Welcome to the Home Page!"),
            ft.ElevatedButton("Go to Settings", on_click=data.go("/settings"))
        ]
    )
```

- `title`: Automatically sets the page title.
- `protected`: If `True`, the router checks if the user is authenticated (using `data.shared["authenticated"]`). If not, redirects to `route_login`.

Dynamic Routes & Query Parameters

You can define dynamic segments using `:param_name`:

```
@app.page("/user/:id")
def user_profile(data: DataSystem) -> ft.View:
    user_id = data.param("id") # Extracts from URL e.g. /user/42 -> 42
    mode = data.query("mode", "view") # Extracts ?mode=edit

    return ft.View(
        route=f"/user/{user_id}",
        controls=[ft.Text(f"Profile {user_id} - Mode: {mode}")]
    )
```

🛡️ Middlewares

Middlewares allow you to execute logic before a page loads. They return a `MiddlewareResult` to either proceed (`next`), redirect (`redirect`), or show a custom view (`view`).

Global Middleware

Applies to every navigation. Useful for logging or global auth guards:

```
from flet_base.router import MiddlewareResult

@app.middleware
def log_navigation(data: DataSystem):
    print(f"Navigating to {data.page.route}")
    return MiddlewareResult.next()
```

Route-specific Middleware

You can attach specific middlewares to single pages:

```
def check_admin(data: DataSystem):
    if not data.shared.get("is_admin"):
        return MiddlewareResult.redirect("/home")
    return MiddlewareResult.next()

@app.page("/admin", middlewares=[check_admin])
def admin_dashboard(data: DataSystem) -> ft.View:
    # ...
```

🧅 Shells (Layouts)

Shells allow you to wrap the returned `ft.View` in common UI containers, like adding an `AppBar` or `NavigationBar`.

```
@app.shell(exclude=["/login"])
def global_appbar(data: DataSystem, view: ft.View) -> ft.View:
    # Adds an AppBar to all views EXCEPT /login
    view.appbar = ft.AppBar(title=ft.Text("My App"))
    return view

@app.shell(prefix="/admin")
def admin_sidebar(data: DataSystem, view: ft.View) -> ft.View:
    # Prepends a custom Navigation block to any route starting with /admin
    view.controls.insert(0, ft.Text("Admin Area Menu"))
    return view
```

🔄 Navigation & Data Sharing

The `DataSystem` instance (`data`) injected into every function is your gateway to controlling the app.

- **`data.go(route)`** : Returns an event handler (lambda) to navigate to a new route. Beautiful for buttons: `on_click=data.go('/home')`
- **`data.page.go(route)`** : Navigates immediately in code.
- **`data.go_back()`** : Navigates to the previous route in history.
- **`data.shared`** : A persistent dictionary to safely store state across pages.
  - Setup session: `data.shared["user"] = "john_doe"`
  - Verify admin status: `data.shared["is_admin"] = True`
  - Get data: `data.shared.get("user")`

Custom 404 Page

Create a custom page for unmatched routes:

```
@app.page_404
def build_404(data: DataSystem) -> ft.View:
    return ft.View(route="/404", controls=[ft.Text("Page not found!")])
```

📦 Summary & Project Structure Example

Flet-base is a premium starter kit that provides a set of reusable, well-integrated modules to accelerate the development of multi-platform applications with **Python** and **Flet**. It includes:

- 🌍 **Translation Manager** -- CSV-based i18n with auto-detection and persistence.
- 🎨 **Theme Manager** -- Light/dark theme toggling with persistent user preference.
- 🧩 **UI Components** -- Pre-styled buttons, texts, data displays, inputs, and modals that automatically adapt to the active theme.
- 📐 **Responsive Auto Layout** -- A container that dynamically arranges children into rows, scales them if needed, and switches to a single column below a configurable threshold.
- 🛤️ **Router Manager** -- Decorator-based routing with dynamic segments, query strings, middlewares, shared state, and layout shells.

All modules are designed to work together seamlessly, allowing you to build polished, production-ready apps with minimal boilerplate.

🗂️ Typical Project Structure

Below is a recommended layout for a project using `flet-base`:

```
my_app/
├── assets/
│   └── translations.csv # All translation strings
├── src/
│   ├── __init__.py
│   ├── main.py # Entry point - config, router, theme setup
│   ├── pages/
│   │   ├── __init__.py
│   │   ├── home.py # Home page view
│   │   ├── profile.py # Profile page with dynamic route
│   │   └── settings.py # Settings page
│   ├── components/ # (Optional) Your own reusable components
│   │   └── custom_card.py
│   └── utils/ # (Optional) Helper functions
├── pyproject.toml (or setup.py)
└── README.md
```

🚀 Minimal `main.py` Example

The following example demonstrates how to initialise the configuration, set up the translation and theme managers, define two pages using the router, and start the application.

```python
import os
import sys

# Allow running from the project root
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import flet as ft
import flet_base.router as fr
from flet_base.config import flet_config
from flet_base.translations import instance_translation_manager as tm
from flet_base.themes import instance_themes as themes
import flet_base.components.buttons as btn
import flet_base.components.texts as txt
import flet_base.components.data_display as dd
import flet_base.components.inputs as inputs

# ========================================================
# GLOBAL CONFIGURATION
# ========================================================

flet_config.translations_csv_path = os.path.join(os.path.dirname(__file__),
"translations.csv")
flet_config.default_theme_mode = "light"

# ========================================================
# CREATE THE ROUTER
# ========================================================

app = fr.FletRouter(
    route_init="/home",
)

# ========================================================
# MIDDLEWARE & SHELLS
# ========================================================

@app.middleware
async def init_systems(data: fr.DataSystem):
    """Initializes themes and translations systems."""
    await tm.awake(data.page)
    await themes.awake(data.page)
    return fr.MiddlewareResult.next()

@app.shell()
def main_shell(data: fr.DataSystem, view: ft.View) -> ft.View:
    """Global shell with Navigation Controls."""

    # Theme toggle icon
```

```
    theme_icon = ft.Icons.DARK_MODE if themes.actual_theme == themes.light_theme else
ft.Icons.LIGHT_MODE

    async def toggle_theme(_):
        await themes.switch_theme(data.page)
        # Re-build the view to apply new theme colors to all components
        data.page.go(data.page.route)

    async def change_lang(e):
        code = tm.get_language_code(e.data)
        await tm.set_language(code, data.page)
        # Force the router to re-build the current view with new translations
        data.page.go(data.page.route)

    view.appbar = ft.AppBar(
        title=txt.title("Flet-base Demo", size=20),
        bgcolor=themes.actual_theme["surface"],
        actions=[
            ft.IconButton(theme_icon, on_click=toggle_theme),
            inputs.dropdown(
                label="Language",
                options=[ft.dropdown.Option(lang) for lang in
tm.get_available_languages()],
                value=tm.get_language_name(tm.active_lang),
                on_change=change_lang,
            ),
        ]
    )
    return view

# ========================================================
# PAGE 1: HOME
# ========================================================

@app.page("/home", title="Home")
def home_page(data: fr.DataSystem) -> ft.View:
    return ft.View(
        route="/home",
        controls=[
            txt.title_primary(tm.translate("welcome")),
            txt.body(f"Current language: {tm.get_language_name(tm.active_lang)}"),
            dd.card(
                content=[
                    txt.subtitle("Package Highlights"),
                    txt.body("- Router-based navigation\n- Theme Management\n- I18n
support\n- Premium Components"),
                ]
            ),
            btn.filled_btn(tm.translate("settings"), icon=ft.Icons.SETTINGS,
```

```
on_click=lambda _: data.page.go("/about")),
        ],
        vertical_alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER,
    )


# ========================================================
# PAGE 2: ABOUT (Used as Settings here)
# ========================================================

@app.page("/about", title="About / Settings")
def about_page(data: fr.DataSystem) -> ft.View:
    return ft.View(
        route="/about",
        controls=[
            txt.title_secondary(tm.translate("settings")),
            dd.icon(ft.Icons.INFO_OUTLINE, size=50),
            dd.progress_bar(value=0.5),
            txt.body("This page demonstrates more components and translated text."),
            btn.text_btn(tm.translate("goodbye"), on_click=lambda _:
data.page.go("/home")),
        ],
        vertical_alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER,
    )


# ========================================================
# 404 PAGE
# ========================================================

@app.page_404
def not_found(data: fr.DataSystem) -> ft.View:
    return ft.View(
        route="/404",
        controls=[
            txt.error_text("404 - Page Not Found", size=30),
            btn.filled_btn("Go Home", on_click=lambda _: data.page.go("/home")),
        ],
        vertical_alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER,
    )


# ========================================================
# RUN THE APP
# ========================================================
```

```python
if __name__ == "__main__":
    app.run()
```

🔍 Key Points

- The global config (`flet_config`) lets you set defaults for translations and themes before importing any manager.
- The translation manager reads strings from `translations.csv` (see the required format in the Translations section).
- The theme manager is automatically used by the components; you can also switch themes manually.
- The router handles navigation, dynamic routes, and middlewares.

The `DataSystem` object passed to every view provides access to:

- Route parameters
- Query strings
- Shared state
- Navigation helpers

Components (buttons, texts, etc.) are **theme-aware**. They fetch colours from the active theme, ensuring visual consistency throughout the app.

This structure keeps your code organised, separates concerns, and lets you leverage the full power of **flet-base** with minimal friction.